A Dependency-Minimized Parallel Computing Architecture Based on the AIKernel Operator Model

A Lean-C-C# Externalized Operator Design for the Prime Phase Generator

Takuya Sogawa

2026-06-02

A Dependency-Minimized Parallel Computing Architecture Based on the AIKernel Operator Model

A Lean-C-C# Externalized Operator Design for the Prime Phase Generator

Author: Takuya Sogawa
Affiliation: AIKernel Project
ORCID: https://orcid.org/0009-0009-7499-2595
Version: v0.1.0
Status: Technical Note / Architecture Draft
Based on: AIKernel.RH v1.5.3
Date: 2026-06-02
DOI: `10.5281/zenodo.20493017`

Canonical version: English (`paper-en.md` / `paper-en.pdf`)
Japanese version: Companion translation (`paper-ja.md` / `paper-ja.pdf`)

Abstract

This technical note presents the **AIKernel Operator Model** as a design discipline for dependency-minimized parallel computation. The case study is the Prime Phase Generator used in AIKernel.RH, a phase-interference-based prime generation and evaluation system.

In this note, "dependency-minimized" does not mean that an entire runtime system has no dependencies. It means that the semantic evaluation of each input unit does not depend on the evaluation result of any other input unit and does not require shared mutable state. Under this constraint, each input value can be treated as an independent Operator invocation, while the Provider layer is free to choose chunking, batching, scheduling, and placement strategies.

Across the AIKernel.RH v1.3.0 to v1.5.3 development line, the phase-interference computation was organized into a three-layer externalized Operator architecture: Lean 4 for formal semantic modeling, C for native pure computation, and C# for Provider-level orchestration. The result is an architecture in which a mathematically motivated computation core is separated from runtime scheduling, resource routing, observation, cancellation, and replay.

The purpose of this note is not to claim superiority over existing prime generation or primality testing methods. Rather, it uses the Prime Phase Generator as a concrete example of how AIKernel can externalize a pure computation as an Operator and execute it through a dependency-minimized, chunked, batched, and parallel Provider model.

Keywords

AIKernel; Operator Model; Interface-Led Architecture; Parallel Computing; Lean 4; Native Operator; Prime Phase Generator; Phase Interference; C#; P/Invoke; Lock-Free Computation; TensorKernel

---

1. Introduction

Modern computing hardware increasingly exposes parallel capacity through multi-core CPUs, SIMD instructions, GPUs, NPUs, and specialized accelerators. However, software architectures do not automatically benefit from this capacity. Shared mutable state, locks, sequential data dependencies, I/O waits, memory bandwidth, scheduling overhead, and result aggregation can all prevent an application from scaling with the available hardware.

The **Operator Model** in AIKernel addresses this issue at the architectural level. An Operator is a semantically closed computation unit. Ideally, it receives an input, returns a deterministic result, does not modify external state, and does not depend on the result of any other Operator invocation.

When a problem can be expressed as a collection of such independent Operator evaluations, the runtime may partition the input space and evaluate each partition in parallel. In AIKernel, this is not treated merely as an implementation trick. It is treated as a contract defined by Interface-Led Architecture: the Operator defines what is computed, the Provider decides how and where it is executed, and the Observer records what happened.

This note uses the Prime Phase Generator as a case study. In this model, the interference energy of a natural number `n` can be evaluated without knowing the evaluation result for `n - 1`, `n + 1`, or any other candidate. This makes the model suitable for chunked and batched execution from C# into a native C Operator.

The central architectural question is therefore:

```
How can a formally motivated pure computation be externalized as a native
Operator and then executed by a Provider with minimal data dependency,
minimal synchronization, and high observability?
```

---

2. Phase-Interference Computation as an Operator

The phase-interference model treats primality as the absence of non-trivial internal interference. A composite number contains internal divisor structure; a prime number, once the domain condition `2 <= n` is applied, contains no non-trivial divisor. This motivates the use of an `interferenceEnergy` function.

At the architectural level, the details of this energy function are less important than its computational shape. The evaluation of one input is independent from the evaluation of another input.

Conceptually, the computation can be expressed as follows:

```
InterferenceOperator : Nat -> InterferenceEnergy
```

A stable-point or prime-candidate test may be represented as:

```
StablePointOperator : Nat -> Bool
```

The Operator contract requires the following properties:

Property	Meaning
Deterministic	The same input produces the same output.

Property	Meaning
Local	The computation depends only on the input value.
Side-effect free	The Operator does not mutate shared runtime state.
Reentrant	Multiple calls can be evaluated independently.
Batchable	A sequence of inputs can be evaluated as a unit.
Observable	Results and metrics can be recorded externally.

This makes the computation close to an **embarrassingly parallel** workload. However, the AIKernel framing is more specific: the independence is elevated into a semantic contract that allows the Provider to change execution order, chunk size, target resource, and degree of parallelism without changing Operator meaning.

---

3. The AIKernel Operator / Provider / Observer Separation

AIKernel separates computation into three roles.

Role	Responsibility
Operator	Defines pure semantic computation.
Provider	Maps Operator execution to runtime resources.
Observer	Collects results, metrics, and replay evidence.

This separation is important because the computation itself should not be entangled with scheduling, logging, thread management, or resource routing. If the Operator remains pure, the Provider can decide whether to run it on a managed .NET implementation, a native C implementation, a SIMD implementation, a GPU implementation, or a future TensorKernel backend.

For the Prime Phase Generator, the Operator evaluates phase-interference energy or stable-point status for each input value. The Provider partitions the search space into chunks and invokes the native Operator in batches. The Observer records throughput, latency, cancellation behavior, result counts, and replayable execution traces.

This separation can be summarized as:

```
Operator: what is computed
Provider: where and how it is executed
Observer: what evidence is preserved
```

The strictness of this boundary is what allows the architecture to remain portable across runtime environments.

---

4. Externalizing the Operator: Lean -> C -> C#

AIKernel.RH v1.3.0 to v1.5.3 introduced a practical externalization path for the Prime Phase Generator. The computation is organized into three layers.

4.1 Lean 4 for Formal Semantics

Lean 4 is used to express the mathematical semantics of the model. It gives a formal account of relationships such as:

```
Nat.Prime n
interferenceEnergy n = 0
isStableFixedPoint n
```

A key boundary condition is that `interferenceEnergy n = 0` alone may also include `0` and `1`. Therefore, the stable fixed point predicate must include a domain condition such as `2 <= n` when it is used as an equivalent characterization of primality over natural numbers.

Lean's role in this architecture is not to manage threads or optimize runtime scheduling. Its role is to define the mathematical contract that the Operator is expected to satisfy.

4.2 C as a Native Operator Layer

The C layer implements the pure computation core as a native Operator. This layer avoids managed allocation overhead and exposes a narrow function boundary suitable for batch execution.

A native Operator should ideally satisfy the following runtime properties:

- it accepts primitive numeric inputs or contiguous buffers;
- it produces deterministic outputs;
- it does not rely on global mutable state;
- it can be called concurrently when the runtime contract permits it;
- it supports batch input in order to reduce boundary-crossing overhead;
- it allows the Provider to control chunk size and parallelism.

In this design, C is not merely an optimization detail. It is the externalized execution form of the Operator.

4.3 C# as the Provider Orchestration Layer

The C# layer, implemented in AIKernel.NET style, acts as the Provider. It does not redefine the mathematical meaning of the computation. Instead, it controls execution.

Typical Provider responsibilities include:

- partitioning the search range into chunks;
- preparing native input buffers;
- controlling the degree of parallelism;
- reducing P/Invoke boundary cost through batching;
- propagating cancellation;
- streaming or aggregating results;
- emitting metrics to Observers;
- preserving replayable execution records.

This gives the architecture a clean boundary: Lean defines the semantics, C evaluates the pure computation, and C# schedules the work.

---

5. Parallelization Strategies

During the AIKernel.RH development process, several strategies were considered for invoking the native Operator from C#.

5.1 SingleCall Parallel

The simplest strategy is to call the native Operator once per input value.

```
for each n in range:
    call native_operator(n)
```

This strategy is easy to implement and directly demonstrates input-level independence. However, if the computation for each input is small, the P/Invoke boundary cost may dominate. The model remains correct, but throughput can be limited by the cost of crossing the managed-native boundary too frequently.

5.2 Batched Execution

The second strategy groups multiple values into one native call.

```
call native_operator_batch(values[])
```

Batching reduces boundary-crossing overhead. It is especially useful when the native Operator is lightweight and the input range is large.

However, a single huge batch can reduce observability and control. Cancellation latency may increase, progress reporting becomes coarser, and memory pressure may become harder to manage.

5.3 Chunked Batched Parallel

The practical strategy reached in the AIKernel.RH v1.5.3 architecture is **Chunked Batched Parallel** execution. The Provider partitions the search range into logical chunks. Each chunk is submitted as a batch to the native Operator, and multiple chunks can be evaluated concurrently.

```
range -> chunks -> batched native calls -> result streams
```

The conceptual execution flow is:

```
for each chunk in partition(range):
    run async:
        call native_operator_batch(chunk)
        emit results
```

This strategy has several advantages:

- fewer P/Invoke calls than SingleCall Parallel;
- better observability than one huge batch;
- chunk-level cancellation and progress reporting;
- independent scheduling of work units;
- reduced need for shared mutable state;
- tunable batch size and parallelism.

The key condition is that chunks are semantically independent. Results may be aggregated later without changing the meaning of each individual Operator evaluation.

---

6. Evaluation Scope

This v0.1.0 note is an architecture draft. It does not yet include a full benchmark suite. Future versions should measure at least the following:

Metric	Purpose
Throughput	Number of evaluated inputs per second.
Boundary cost	Cost difference between single and batched calls.
Batch sensitivity	Throughput changes as batch size changes.
Parallel efficiency	Scaling behavior as worker count increases.
Memory pressure	Allocation and buffer pressure during execution.
Cancellation latency	Time required to stop chunk-level execution.
Observer overhead	Cost of metrics and result collection.
Replay overhead	Cost of preserving audit-ready traces.

Metric	Purpose

The working hypothesis is:

```
For Operators with no input-to-input data dependency,
Chunked Batched Parallel execution can reduce boundary cost
relative to SingleCall Parallel execution while retaining
better observability and control than a single huge batch.
```

This hypothesis is architectural, not a benchmark result. It must be validated by future measurements.

---

7. Discussion

7.1 What "Dependency-Minimized" Means

The phrase "dependency-minimized" must be interpreted precisely. It does not mean that the entire system has no dependencies. The Provider depends on the operating system, scheduler, memory, native libraries, runtime configuration, and hardware. The Observer depends on logging, metrics, and storage.

The claim is narrower:

```
The semantic evaluation of one input value does not depend on
the semantic evaluation result of another input value.
```

This is the property that enables free reordering, partitioning, batching, and parallel scheduling.

7.2 Relationship to Amdahl's Law

This architecture does not invalidate Amdahl's law. Sequential and shared components remain: input generation, chunk management, boundary crossing, result aggregation, memory bandwidth, and observation.

The value of the Operator model is that it localizes these sequential components outside the pure computation. By separating Operator, Provider, and Observer, the architecture maximizes the portion of the workload that can be executed independently.

In this sense, AIKernel does not remove all runtime constraints. It removes unnecessary semantic dependencies from the computation contract.

7.3 Generality Beyond Prime Generation

The Prime Phase Generator is a case study, not the only target. The same architecture may apply to any workload that satisfies the following conditions:

1. each input can be evaluated independently;
2. the evaluation is deterministic;
3. the output contract is clear;
4. batching is possible;
5. results can be safely aggregated;
6. observation can be separated from computation.

Potential application domains include:

- independent SAT assignment evaluation;
- candidate-key search in cryptographic analysis;
- candidate-divisor evaluation in factorization workflows;
- large parameter sweeps;

- independent simulation samples;
- batch rule validation;
- deterministic preprocessing for AI governance pipelines.

Each domain still requires its own proof of independence. AIKernel does not assume independence; it makes independence an explicit Operator contract.

---

8. Toward TensorKernel

A long-term direction of the AIKernel project is **TensorKernel**: a routing layer for executing pure Operators across heterogeneous compute resources such as CPU, SIMD, GPU, NPU, distributed nodes, or specialized accelerators.

If an Operator has a stable semantic contract, a Provider can choose the execution backend without changing the meaning of the computation.

Resource	Suitable workload shape
CPU	Branch-heavy integer computation.
SIMD	Repeated homogeneous numeric evaluation.
GPU	Massive data-parallel computation.
NPU	Tensorized approximate evaluation.
Distributed nodes	Large partitionable search spaces.
Quantum accelerator	Special problems mapped to QUBO or related forms.

The Prime Phase Generator demonstrates the first step in this direction: a formally motivated computation is externalized as a pure Operator and then orchestrated by a Provider.

---

9. Non-Claims

To avoid overstatement, this note explicitly does not claim the following:

1. It does not claim performance superiority over existing prime generation or primality testing methods.
2. It does not claim that every computation can be decomposed into dependency-free Operators.
3. It does not claim to invalidate Amdahl's law.
4. It does not claim that the C implementation is fully automatically extracted from Lean.
5. It does not claim completed GPU, NPU, or quantum implementations.
6. It does not claim absolute benchmark performance before measurements are published.

The claim is architectural: for computations whose input units are semantically independent, the AIKernel Operator Model provides a disciplined way to externalize the pure computation and execute it through a chunked, batched, observable Provider.

---

10. Conclusion

This note presented a dependency-minimized parallel computation architecture based on the AIKernel Operator Model. Using the Prime Phase Generator as a case study, it showed how a formally motivated computation can be organized into a Lean-C-C# externalized Operator pipeline.

Lean provides the formal semantic contract. C provides the native pure computation. C# provides Provider-level orchestration, including chunking, batching, scheduling, cancellation, observation, and replay.

The resulting architecture separates meaning from execution. It does not rely on a single implementation strategy, and it does not require the computation to be tied to a specific runtime. Instead, it allows a pure computation to be routed through different Providers while preserving its semantic contract.

Future work should include benchmark measurements, batch-size tuning, Observer-cost analysis, GPU/NPU Provider exploration, and integration with the broader TensorKernel design.

Appendix A. Minimal Architecture Sketch

```
+-----------------------------+
| Lean 4 Formal Model         |
| - Prime equivalence         |
| - Energy zero semantics     |
| - Stable fixed point proof  |
+--------------+--------------+
               |
               v
+-----------------------------+
| Native C Operator           |
| - Pure computation core     |
| - Batch input               |
| - Deterministic output      |
+--------------+--------------+
               |
               v
+-----------------------------+
| C# AIKernel Provider        |
| - Chunk partitioning        |
| - Parallel scheduling       |
| - P/Invoke boundary control |
| - Cancellation              |
+--------------+--------------+
               |
               v
+-----------------------------+
| Observer / Replay / Metrics |
| - Result collection         |
| - Throughput measurement    |
| - Audit trail               |
+-----------------------------+
```

Appendix B. Provider-Level Execution Sketch

```
public async Task<IReadOnlyList<PrimeCandidateResult>>
    EvaluateRangeAsync(
        ulong start,
        ulong end,
        int chunkSize,
        int maxDegreeOfParallelism,
        CancellationToken cancellationToken)
{
    var chunks = PartitionRange(start, end, chunkSize);
```

```
    var options = new ParallelOptions
    {
        MaxDegreeOfParallelism = maxDegreeOfParallelism,
        CancellationToken = cancellationToken
    };

    var results = new ConcurrentBag<PrimeCandidateResult>();

    await Parallel.ForEachAsync(
        chunks,
        options,
        async (chunk, ct) =>
        {
            var input = CreateBatch(chunk);

            var output =
                NativePrimePhaseOperator.EvaluateBatch(input);

            foreach (var item in output)
            {
                results.Add(item);
            }

            await Task.CompletedTask;
        });

    return results.ToArray();
}
```

This code is a conceptual sketch. A production implementation should handle native buffer ownership, allocation reduction, exception translation, cancellation propagation, metrics emission, replay logging, and result ordering more carefully.

References

Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. AFIPS Conference Proceedings, 30, 483-485.
DOI: 10.1145/1465482.1465560

Gustafson, J. L. (1988). Reevaluating Amdahl's Law. Communications of the ACM, 31(5), 532-533.
DOI: 10.1145/42411.42415

de Moura, L., Kong, S., Avigad, J., van Doorn, F., & von Raumer, J. (2015). The Lean Theorem Prover. Automated Deduction - CADE-25, Lecture Notes in Computer Science, 9195, 378-388. Springer.
DOI: 10.1007/978-3-319-21401-6_26

The mathlib Community. (2020). The Lean mathematical library. Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, 367-381.
DOI: 10.1145/3372885.3373824

Sogawa, T. (2026). Phase-Interference Energy and the Formal Structure of the PG1224 Prime Generation System: A Lean 4 Formalization of Prime = Energy 0 = Stable Fixed Point. Zenodo.
DOI: 10.5281/zenodo.20483437

Sogawa, T. (2026). AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference. Zenodo.
DOI: 10.5281/zenodo.20290614

Sogawa, T. (2026). AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems. Zenodo.
DOI: 10.5281/zenodo.20460322